WhoAreWe?

By Raymond DiDonato, Gabriel Ball, Samuel Belanger and Charrette--Hubinois Lohann.

Concept :

'WhoAreWe?' is a web application with the goal of helping members of a group remember each other's name. It was a request of Lohann's father who meets a lot of people and has a hard time remembering their names, and built according to his specifications :

> Easily enter the following :
>
> - Picture
> - Full name
> - Role in the company / job
> - District / department
> - Location (ranging from office number to city)
>
> Then be able to answer multiple choice questions / open questions. Get points based on speed and negative points on wrong answers, with an option to skip questions.
> Have a leaderboard of your own scores, let the admins enable a public leaderboard with everyone's scores.
> Have the data be internally and securely stored, and the app managed by (in case of a company) HR

After discussing the features together, we chose which features to implement. We decided to cut non essential features like open ended questions, and time based point system because of time constraints, and thanks to that, we now have a working prototype with all the necessary ones.

The Team :

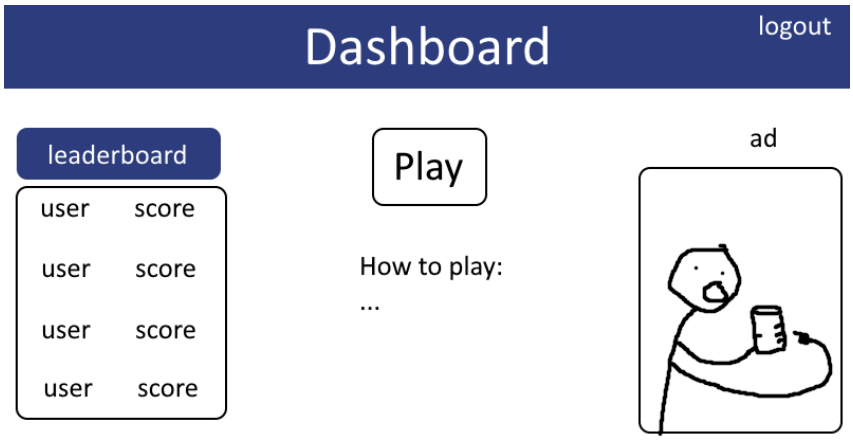
- Raymond DiDonato
  - Play screen
  - UML Diagrams
- Gabriel Ball
  - Administrator (HR) Dashboard - Frontend CRUD on users
- Samuel Belanger
  - Database
  - CRUD API scripts
- Charrette--Hubinois Lohann
  - Creative Lead
  - Dashboard
  - CSS & Miscellaneous

Technical summary:

For this project, our team of 4 built a working full-stack application to assist employees of large companies with remembering things about their coworkers. The app uses a front-end built with React, a backend API written in PHP, and a MySQL database for persistent storage.

UI Mockup :

Dashboard page:



Quiz page:

Who is this?

What is their name?

a b

c d

What is their role?

a b

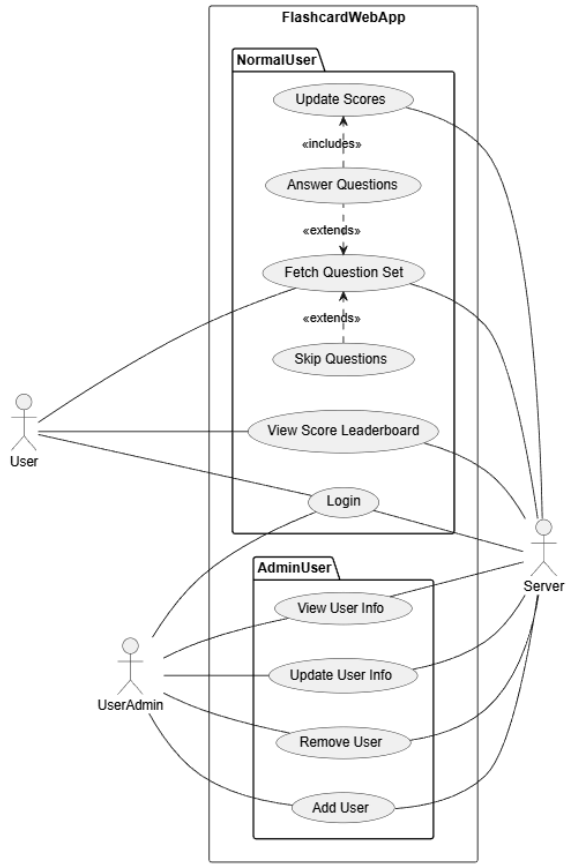
c d

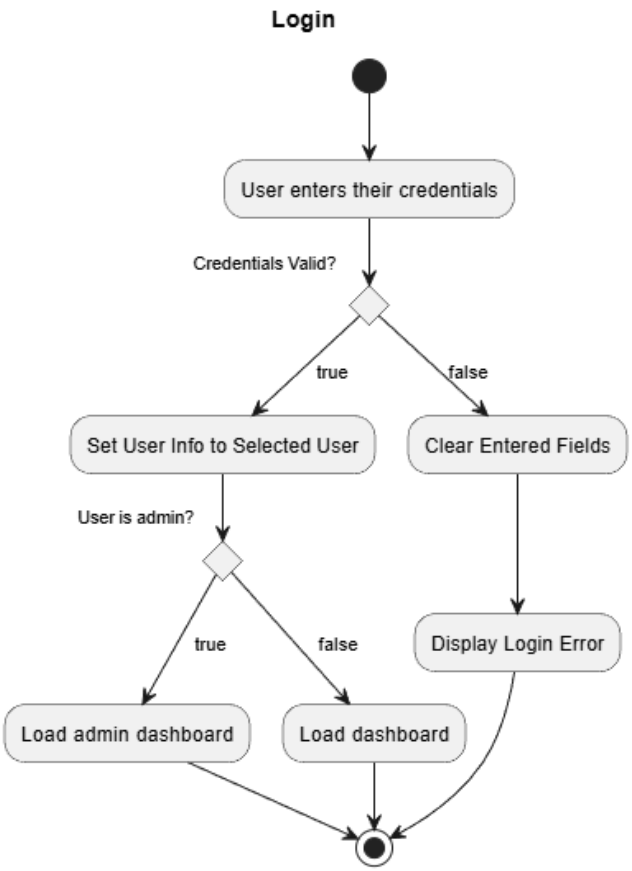
.
.
.

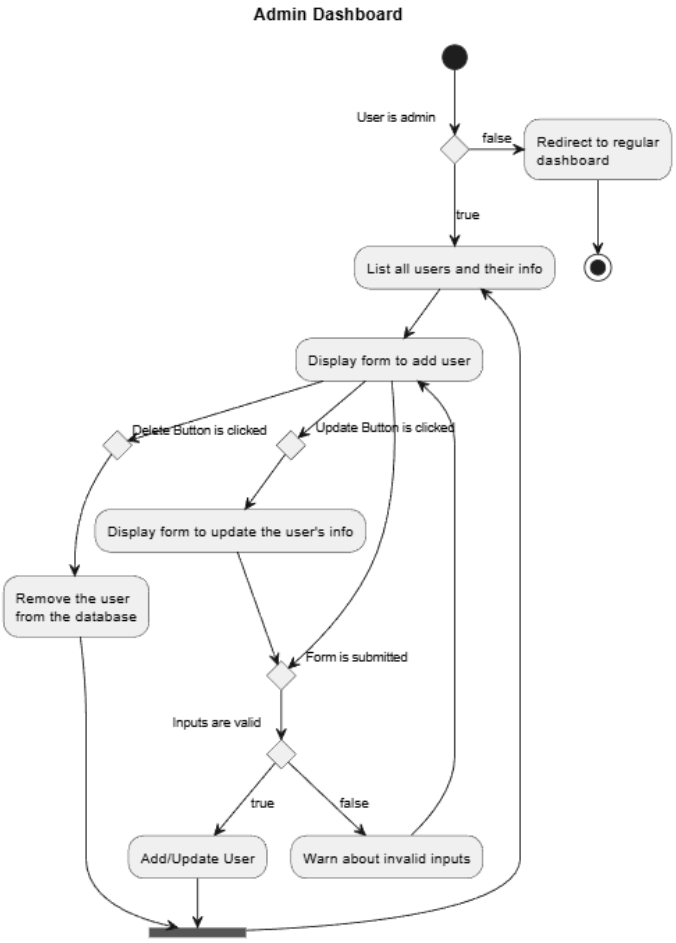
skip submit

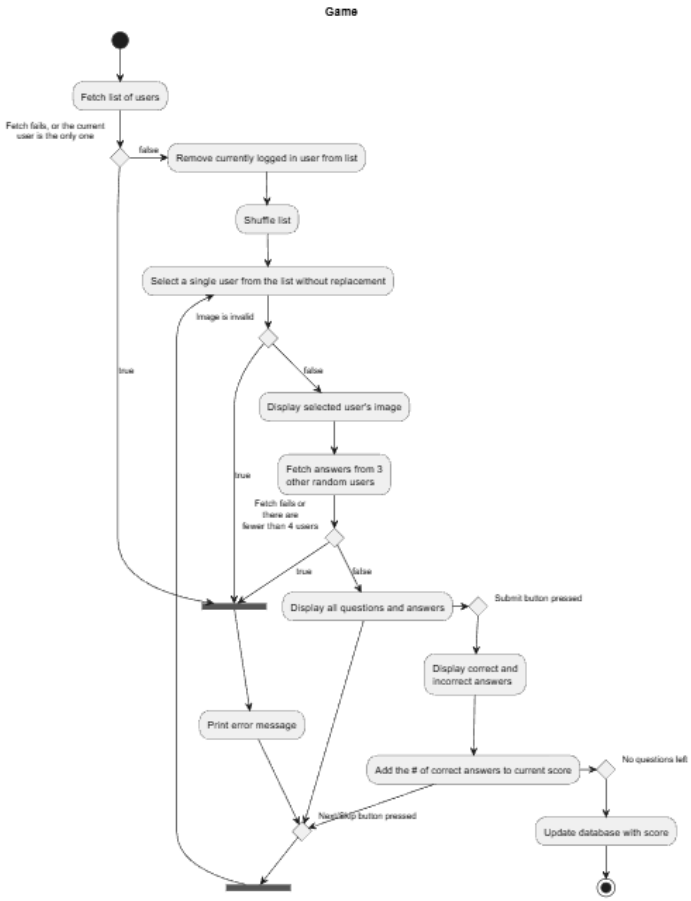
System Components:

UML:






Admin Dashboard
User is admin
false
Redirect to regular dashboard
true
List all users and their info
Display form to add user
Delete Button is clicked
Update Button is clicked
Display form to update the user's info
Remove the user from the database
Form is submitted
Inputs are valid
true
false
Add/Update User
Warn about invalid inputs


Game
Fetch list of users
Fetch fails, or the current user is the only one
false
Remove currently logged in user from list
Shuffle list
Select a single user from the list without replacement
Image is invalid
true
false
Display selected user's image
Fetch answers from 3 other random users
true
Fetch fails or there are fewer than 4 users
true
false
Display all questions and answers
Submit button pressed
Display correct and incorrect answers
Print error message
Add the # of correct answers to current score
No questions left
Next/Skip button pressed
Update database with score

API:

- **Login.php**
  - receive user id and password, checks password received with stored password, returns the user id on success
- **Register.php**
  - Receives a new user id, password, name, and any additional info. If user isn't taken, creates a new user with that info in the database\
- **Delete.php**
  - Deletes the user with a given id
- **Get_user.php**
  - Returns all information except password for a given user id
- **List_users.php**
  - Returns all information except password for all users
- **List_scores.php**
  - Returns only user names and scores for all users
- **Update_info.php**
  - Allows updating user information used for quiz questions
- **Update_password.php**
  - Takes the old password and a new password to update a user's password
- **Update_score.php**
  - Updates a users score after finishing a quiz
- **Get_answers.php**
  - Returns unique information on 3 random users that *don't* match a given id. Used by the quiz to dynamically gather wrong answers

Frontend:

- **App.jsx**
  - To establish routes (ex: /admin_dashboard or /play)
- **RequireAuth.jsx**
  - To ensure only logged-in users can go beyond the login page, and that only admin users can access admin pages
- **Session.js**
  - To store the currently logged-in user in localStorage
- **Play.jsx**
  - Page to play the quiz

- **Login.jsx** and **LoginForm.jsx**
    - Page for logging in as either regular user or admin user
- **Dashboard.jsx**
    - Page for regular users (as well as admins) to see the leaderboard, tutorial, and access to the Play page
- **Tutorial.jsx**
    - To inform users how to play the quiz
- **AdminDashboard.jsx**
    - Page for admin users to see a list of all users, create new users, and delete existing users, with the ability to edit a select user on a separate EditUser page
- **Header.jsx**
    - To let users know what page they're on and give them the ability to logout
- **EditUser.jsx**
    - Page to let admin users edit the details of a select user, as well as change their password
- **UserForm.jsx**
    - To let admin users create a new user or edit an existing user
- **ChangePassword.jsx**
    - To let admin users change the password of a select user
- **UserList.jsx** and **UserListItem.jsx**
    - Shows admin users a full list of existing users in the form of a table
- **QuestionSet.jsx** and **Question.jsx**
    - To present the user with the questions they are to answer for the quiz
- **Leaderboard.jsx**
    - To show users the leaderboard as it stands
- **FakeAd.jsx**
    - To simulate ads for visual symmetry

Lessons Learned

All members learned the value of establishing the requirements of an application early on its development, and experienced the difficulties in syncing changes between the frontend and the backend.

Sam learned:

- How to organize a full-stack project in a git repository
- How to make a PHP REST API
- How to use PHP to communicate with a database

Gabe learned:

- React best practices
- How to send to and receive data from backend
- How to set up multi-page React application using Routes

Raymond learned:

- React/JS best practices
- The importance of graceful async and error handling while making requests to an API (to avoid spamming the API)
- Various PlantUML positioning quirks

Lohann learned:

- MySQL database using PHP basics
- Client to requirements translation.
- More css :(

Improvements and ideas for the future :

- Implement the cut features and get feedback from the client
- Allow the HR to upload pictures for the member directly in the create user
- Turn the app from instance based to server based
    - Instead of having an instance per group, store instances akin to a discord server and allow users to create their own instances